Astronomer Cosmos Setup

sanima-bank-dbt on Airflow 3.3.2 (Docker Compose)

1. Do I need to install dbt-core?

Short answer: no, not separately. `dbt-postgres` lists `dbt-core` as a dependency, so pip installs dbt-core automatically when you install the adapter. The Dockerfile line below gives you both:

```
/opt/airflow/dbt_venv/bin/pip install --no-cache-dir dbt-postgres
```

You can list it explicitly if you want to pin versions so every build is identical, for example:

```
/opt/airflow/dbt_venv/bin/pip install --no-cache-dir "dbt-core==1.10.*" "dbt-postgres==1.10.*"
```

Explicit listing is optional. It does not change behaviour, it only controls which version you get.

Why dbt lives in its own virtual environment

Airflow and dbt both depend on many of the same libraries (protobuf, jinja, etc.) and often need different versions. Putting dbt in `/opt/airflow/dbt_venv` keeps them apart. Cosmos is told where the dbt binary is through `ExecutionConfig(dbt_executable_path="/opt/airflow/dbt_venv/bin/dbt")`.

Package	Installed into	Why
astronomer-cosmos	Airflow's Python	Airflow must import it to build the DAG
dbt-postgres	/opt/airflow/dbt_venv	Runs your dbt models against Postgres
dbt-core	/opt/airflow/dbt_venv	Pulled in automatically by dbt-postgres

2. What does `docker compose build` need?

`docker compose build` builds a custom image for every service that has a `build:` key. In this project all Airflow services (api-server, scheduler, dag-processor, worker, triggerer, init, cli) share the `x-airflow-common` block, which now contains `build: .`, so they all get the same image.

What it needs to work

- **A Dockerfile** in the same folder as `docker-compose.yaml` (`build: .` means "this folder"). Already created.
- **Docker Desktop running**.
- **Internet access**, to pull `apache/airflow:3.3.2` and download packages from PyPI.

What it does, step by step

- Starts from `apache/airflow:3.3.2`.
- Installs `astronomer-cosmos` into Airflow's Python. Pinning `apache-airflow==${AIRFLOW_VERSION}` in the same command stops pip from upgrading or downgrading Airflow by accident.
- Creates `/opt/airflow/dbt_venv` and installs `dbt-postgres` (plus dbt-core) into it.
- Saves the result as a local image that `docker compose up` uses.

The Dockerfile

```
FROM apache/airflow:3.3.2

# Cosmos runs inside Airflow's own Python environment
RUN pip install --no-cache-dir "apache-airflow==${AIRFLOW_VERSION}" astronomer-cosmos

# dbt goes into a separate venv so its dependencies can't clash with Airflow's
RUN python -m venv /opt/airflow/dbt_venv && \
    /opt/airflow/dbt_venv/bin/pip install --no-cache-dir dbt-postgres
```

Rebuild whenever you change the Dockerfile. Editing DAGs or dbt models does *not* need a rebuild, because those folders are mounted into the containers.

3. Changes made to docker-compose.yaml

All changes are inside the shared `x-airflow-common` block, so they apply to every Airflow service. Nothing else in the file was touched.

Change 1: build the custom image instead of pulling the stock one

```
# BEFORE
  image: ${AIRFLOW_IMAGE_NAME:-apache/airflow:3.3.2}
  # build: .

# AFTER
  # image: ${AIRFLOW_IMAGE_NAME:-apache/airflow:3.3.2}
  build: .
```

The stock image has no Cosmos or dbt. `build: .` makes Compose use the Dockerfile instead.

Change 2: an Airflow connection for the dbt database

```
  environment:
    ...
    AIRFLOW_CONN_SANIMA_POSTGRES: '{"conn_type": "postgres",
        "host": "host.docker.internal", "port": 5432,
        "login": "postgres", "password": "postgres", "schema": "postgres"}'
```

Any environment variable named `AIRFLOW_CONN_<ID>` becomes an Airflow connection, so this creates `sanima_postgres`. Cosmos reads it through `PostgresUserPasswordProfileMapping` and writes a dbt profile at runtime. The values match your `profiles.yml`, except the host: inside a container, `localhost` is the container itself, so `host.docker.internal` is used to reach the Postgres on your Mac. ("schema" here is the database name, in Airflow's naming.)

Change 3: mount the dbt project into the containers

```
  volumes:
    - ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
    - ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
    - ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
    - ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
    - ${AIRFLOW_PROJ_DIR:-.}/sanima-bank-dbt:/opt/airflow/dbt/sanima-bank-dbt   # NEW
```

Airflow can only see files inside the container. This makes your dbt project available at `/opt/airflow/dbt/sanima-bank-dbt`, the path used in the DAG. Edits on your Mac show up immediately.

Change 4: make host.docker.internal resolvable

```
  extra_hosts:
    - "host.docker.internal:host-gateway"
```

Docker Desktop on macOS already provides this name. The line makes the same setup also work on Linux, where it is not defined by default.

4. The DAG: dags/dbt_sanima_cosmos.py

```
profile_config = ProfileConfig(
    profile_name="dbt_proj",
    target_name="dev",
    profile_mapping=PostgresUserPasswordProfileMapping(
        conn_id="sanima_postgres",
        profile_args={"schema": "public"},
    ),
)

sanima_bank_dbt = DbtDag(
    dag_id="sanima_bank_dbt",
    project_config=ProjectConfig("/opt/airflow/dbt/sanima-bank-dbt"),
    profile_config=profile_config,
    execution_config=ExecutionConfig(
        dbt_executable_path="/opt/airflow/dbt_venv/bin/dbt"),
    render_config=RenderConfig(test_behavior=TestBehavior.AFTER_EACH),
    operator_args={"install_deps": False},
    schedule=None,
    ...
)
```

- Each dbt seed, snapshot and model becomes its own Airflow task, in dependency order.
- `TestBehavior.AFTER_EACH`: a model's tests run right after that model.
- `install_deps=False`: the project has no packages.yml, so `dbt deps` is skipped.

5. Running it

```
cd "acedemy airflow"
docker compose down
docker compose build
docker compose up -d
```

Open `http://localhost:8080`, unpause **sanima_bank_dbt** and trigger it.

Troubleshooting

- **Connection refused:** your local Postgres must be running and accept TCP connections on 5432.
- **Different credentials or database:** edit `AIRFLOW_CONN_SANIMA_POSTGRES`, then run `docker compose up -d` (no rebuild needed).
- **DAG doesn't appear:** check the dag-processor logs with `docker compose logs airflow-dag-processor`.